Автономная некоммерческая организация высшего образования
«Санкт-Петербургский институт экономики и управления»

Кафедра 09.02.07

ПРАКТИЧЕСКАЯ РАБОТА № 4

Дисциплина МДК 03.02

Измерение метрик качества кода

Вариант № 2
АИС «Умный кампус»
Модуль Library System

Студент: Шваб Роберт Эдуардович
Группа: И13_24_11
Подпись: _____

Руководитель: _____
Дата представления: «___» _____ 2026 г.

Санкт-Петербург
2026 г.

СОДЕРЖАНИЕ

1 ВЫПОЛНЕНИЕ РАБОТЫ.....3

1.1 Методика измерений.....3

1.2 Запахи кода и рефакторинг.....4

1.3 Проверка результата.....5

1.4 Выводы.....5

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ.....6

ПРИЛОЖЕНИЕ А Листинг исходного кода.....7

1 ВЫПОЛНЕНИЕ РАБОТЫ

Я выполнил работу по варианту 2 Library System АИС «Умный кампус» на C# и .NET 8. Модуль принимает PDF/EPUB/ZIP в карантин; права Reader и Librarian разделены [2]. Оформление выполнено по положению института [1].

1.1 Методика измерений

Я выполнил расчёт одним Roslyn-анализатором до и после исправления. CC начинается с 1 и увеличивается за if, циклы, catch, условное выражение, case и логические &&, ||, ??. LOC — число непустых строк тела, включая скобки; отдельные строки комментариев // исключаются. Это явно заданная методика для сравнения, а не скрытая оценка «на глаз».

Индекс MI рассчитан по нормализованной формуле Microsoft. Расчёт разделён на выражения (1) и (2), объём Холстеда приведён в (3) [3].

$$MI = \max(0, \min(100, A \times 100 / 171)) \quad (1)$$

$$A = 171 - 5{,}2 \ln V - 0{,}23\,CC - 16{,}2 \ln LOC \quad (2)$$

$$V = N \times \log_2 n \quad (3)$$

Здесь N — число токенов тела; n — словарь операторов и операндов. Идентификаторы, строки и числа считаются операндами, остальные токены — операторами. CC и LOC определены выше.

AST-зависимости — число различных имён типов в выбранных позициях синтаксического дерева после исключения служебного списка. Это приближение сцепления, а не семантический CBO. LCOM1 = max(0, P − Q), где P — пары методов без общего поля, Q — пары с общим полем. Учитываются явные поля и используемые параметры первичного конструктора. Для единственного метода LCOM1 равен 0 по определению.

Сведения представлены в таблице 1.

Таблица 1 — Метрики до и после

Показатель	До	После
Строки метода	21	6
Цикломатическая сложность	6	1
Halstead Volume	788.47	375.08
Maintainability Index	50.07	64.87
AST-зависимости класса	2	3
LCOM1 класса	0	0

Источник: составлено автором по результатам работы

Сведения представлены в таблице 2.

Таблица 2 — Методы контроллера каталога

Метод	Строки	CC	V	MI
Get	8	5	709.05	59.67
Download	7	3	385.74	63.05

Источник: составлено автором по результатам работы

BooksController имеет AST-зависимостей 3, LCOM1 — 0. Оба метода обращаются к db; общий захваченный параметр первичного конструктора учтён как состояние класса. Контроллер каталога не подвергался сравнению с исходным обработчиком загрузки: это отдельная область ответственности.

1.2 Запахи кода и рефакторинг

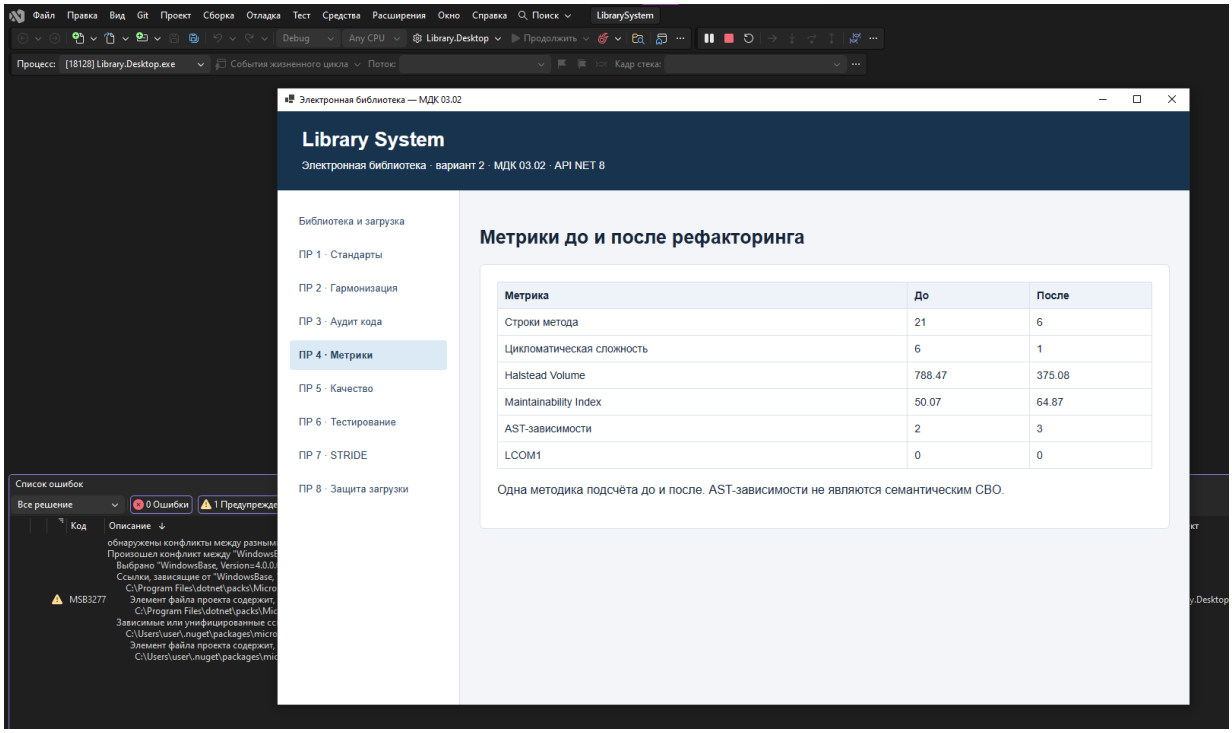
Исходный метод одновременно проверял роль, разбирал архив, создавал каталоги и записывал файлы. Разные ошибки сводились к строке error. Я вынес проверку формата в FilePolicy, полей — в MetadataValidator, транзакцию и файловую запись — в UploadService. Контроллер получает principal, вызывает сервис и формирует ответ.

Уменьшение CC относится к контроллеру: проверки не исчезли, а получили самостоятельные компоненты и тесты. AST-зависимости выросли с 2 до 3 из-за явных HTTP-типов. LCOM1 остался 0; для малого класса он малоинформативен. Поэтому я не объявляю любую метрику улучшенной только ради красивого результата.

1.3 Проверка результата

Ключевой метод сократился с 21 до 6 строк, CC — с 6 до 1; MI вырос с 50.07 до 64.87, на 14.80 пункта. Тест RefactoredMethodHasLowerComplexityAndHigherMi подтверждает направление изменения; отдельный тест if и && проверяет правило CC.

На рисунке 1 показано сравнение метрик исходного и исправленного метода загрузки.



Источник: снимок прогона на Windows, предоставленный автором.

Рисунок 1 — Сравнение метрик метода загрузки до и после рефакторинга

1.4 Выводы

Я получил воспроизводимое сравнение и выполнил рефакторинг с разделением обязанностей. Результат сопровождается методикой и исходным листингом, поэтому увеличение AST-зависимостей и ограничения LCOM1 не скрыты.

Полные исходники, входные данные и журналы переданы с отчётом. Основной листинг приведён в приложении А.

Исходный код и порядок запуска: репозиторий MDK0302-LibrarySystem.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Положение об оформлении письменных работ. Санкт-Петербургский институт экономики и управления. Методические материалы [1].

2. Практические работы МДК 03.02. Задания ПР 1–8 и эссе. Вариант 2 Library System. Материалы преподавателя [2].

3. Microsoft. Code metrics. Maintainability index range and meaning. — URL: https://learn.microsoft.com/en-us/visualstudio/code-quality/code-metrics-maintainability-index-range-and-meaning?view=visualstudio (дата обращения: 08.11.2026) [3].

ПРИЛОЖЕНИЕ А
Листинг исходного кода

before/LegacyUploadController.cs

```
using System.IO.Compression;
namespace Library.Before;
// Изолированный исходный образец. Не включается в сборку API.
public class LegacyUploadController
{
   private string Root = "App_Data";
   public string upload(string role, string name, byte[] bytes)
   {
      if (role == "Librarian" || role == "Reader")
      {
         if (bytes.Length == 0) return "error";
         if (name.EndsWith(".zip"))
         {
            using var zip = new ZipArchive(new MemoryStream(bytes));
            foreach (var entry in zip.Entries)
            {
               string path = Path.Combine(Root, entry.FullName);
               Directory.CreateDirectory(Path.GetDirectoryName(path)!);
               using var input = entry.Open();
               using var output = File.Create(path);
               input.CopyTo(output);
            }
         }
         else File.WriteAllBytes(Path.Combine(Root, name), bytes);
         return "ok";
      }
      return "error";
   }
   public int count() => Directory.GetFiles(Root).Length;
}
```

src/Library.Api/UploadController.cs

```
using System.Security.Claims;
using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Mvc;
using Microsoft.AspNetCore.RateLimiting;
namespace Library.Api;


/// <summary>Принимает книги только от библиотекаря.</summary>
[ApiController]
[Route("api/books")]
[Authorize(Policy = "LibrarianOnly")]
public sealed class UploadController(UploadService service) : ControllerBase
{
   /// <summary>Создаёт записи карантина и возвращает 201.</summary>
   [HttpPost("upload")]
   [EnableRateLimiting("uploads")]
   [RequestSizeLimit(2 * 1024 * 1024 + 65536)]
   public async Task<IActionResult> Upload([FromForm] string title, IFormFile file, CancellationToken ct)
   {
      string actor = User.FindFirstValue(ClaimTypes.NameIdentifier)!;
```

```
        var rows = await service.SaveAsync(title, file, actor, ct);
        return StatusCode(201, rows.Select(x => new { x.Id, x.Title, x.OriginalName,
            x.Length, x.Sha256, Status = "Quarantined" }));
    }
}
```

src/Library.Tools/CodeQuality.cs

```
using System.Text.RegularExpressions;
using Microsoft.CodeAnalysis;
using Microsoft.CodeAnalysis.CSharp;
using Microsoft.CodeAnalysis.CSharp.Syntax;
namespace Library.Tools;
public sealed record MethodMetric(string Name, int Lines, int Complexity, double Volume, double Mi);
public sealed record ClassMetric(string Name, MethodMetric[] Methods, int Coupling, int Lcom1);
public sealed record Finding(string Rule, string Location, string Status, string Description);
/// <summary>Воспроизводимый синтаксический анализ Roslyn; не семантический CBO.</summary>
public static class CodeQuality
{
    public static ClassMetric Measure(string source, string name)
    {
        var root = CSharpSyntaxTree.ParseText(source).GetCompilationUnitRoot();
        var type = root.DescendantNodes().OfType<ClassDeclarationSyntax>().Single(x => x.Identifier.ValueText
== name);
        var methods = type.Members.OfType<MethodDeclarationSyntax>().ToArray();
        var ignored = new HashSet<string> { "Task", "List", "Stream", "Guid", "CancellationToken", "Exception" };
        int coupling = type.DescendantNodes().OfType<TypeSyntax>()
            .Where(x => x.Parent is ParameterSyntax or VariableDeclarationSyntax or
ObjectCreationExpressionSyntax or BaseTypeSyntax)
            .Select(x => x.ToString()).Where(x => x.Length > 0 && char.IsUpper(x[0]) && !ignored.Contains(x) &&
x != name).Distinct().Count();
        var fields = type.Members.OfType<FieldDeclarationSyntax>().SelectMany(x => x.Declaration.Variables)
            .Select(x => x.Identifier.ValueText).ToHashSet();
        if (type.ParameterList is not null)
            fields.UnionWith(type.ParameterList.Parameters.Select(x => x.Identifier.ValueText));
        var access = methods.Select(x => x.DescendantNodes().OfType<IdentifierNameSyntax>()
            .Select(y => y.Identifier.ValueText).Where(fields.Contains).ToHashSet()).ToArray();
        int shared = 0, separate = 0;
        for (int i = 0; i < access.Length; i++) for (int j = i + 1; j < access.Length; j++)
            if (access[i].Overlaps(access[j])) shared++; else separate++;
        return new(name, methods.Select(Method).ToArray(), coupling, Math.Max(0, separate - shared));
    }
    public static MethodMetric Method(MethodDeclarationSyntax method)
    {
        int cc = 1;
        foreach (var node in method.DescendantNodes())
        {
            if (node is IfStatementSyntax or ForStatementSyntax or ForEachStatementSyntax or
                WhileStatementSyntax or DoStatementSyntax or CatchClauseSyntax or
                ConditionalExpressionSyntax or CaseSwitchLabelSyntax) cc++;
            if (node is BinaryExpressionSyntax b && (b.IsKind(SyntaxKind.LogicalAndExpression) ||
                b.IsKind(SyntaxKind.LogicalOrExpression) || b.IsKind(SyntaxKind.CoalesceExpression))) cc++;
        }
        SyntaxNode body = (SyntaxNode?)method.Body ?? (SyntaxNode?)method.ExpressionBody ?? method;
        int lines = body.ToString().Split('\n').Count(x => x.Trim().Length > 0 && !x.TrimStart().StartsWith("//"));
        var tokens = body.DescendantTokens().ToArray();
        bool Operand(SyntaxToken t) => t.IsKind(SyntaxKind.IdentifierToken) ||
t.IsKind(SyntaxKind.StringLiteralToken) || t.IsKind(SyntaxKind.NumericLiteralToken);
        int n = tokens.Where(Operand).Select(x => x.Text).Distinct().Count() + tokens.Where(x => !
Operand(x)).Select(x => x.Text).Distinct().Count();
        double volume = tokens.Length * Math.Log2(Math.Max(2, n));
```

```
      double mi = Math.Clamp((171 - 5.2 * Math.Log(Math.Max(1, volume)) - .23 * cc - 16.2 *
Math.Log(Math.Max(1, lines))) * 100 / 171, 0, 100);
      return new(method.Identifier.ValueText, lines, cc, Math.Round(volume, 2), Math.Round(mi, 2));
   }
   public static Finding[] Audit(string source)
   {
      var root = CSharpSyntaxTree.ParseText(source).GetCompilationUnitRoot();var list = new List<Finding>();
      foreach (var m in root.DescendantNodes().OfType<MethodDeclarationSyntax>().Where(x =>
x.Modifiers.Any(SyntaxKind.PublicKeyword)))
      {
         string name = m.Identifier.ValueText;
         if (!Regex.IsMatch(name, "^[A-Z][A-Za-z0-9]*$")) list.Add(new("AUD-01", name, "Не соответствует",
"Имя метода не PascalCase"));
         if (!m.GetLeadingTrivia().ToFullString().Contains("<summary>")) list.Add(new("AUD-02", name,
"Требует доработки", "Нет XML summary"));
         if (Method(m).Complexity > 10 || Method(m).Lines > 30) list.Add(new("AUD-03", name, "Требует
доработки", "Сложный или длинный метод"));
      }
      if (source.Contains("Path.Combine(Root, entry.FullName)")) list.Add(new("AUD-04", "ZIP", "Не
соответствует", "Непроверенный путь"));
      if (source.Contains("role == \"Reader\"")) list.Add(new("AUD-05", "role", "Не соответствует", "Reader
допущен к записи"));
      if (source.Contains("private string Root")) list.Add(new("AUD-06", "Root", "Не соответствует",
"Приватное поле не _camelCase"));
      if (!source.Contains("Authorize")) list.Add(new("AUD-07", "Controller", "Не соответствует", "Нет
серверной политики"));
      return list.ToArray();
   }
}
```

src/Library.Api/Storage.cs

```
using System.Security.Cryptography;
using FluentValidation;
using Library.Core;
using Microsoft.EntityFrameworkCore;
namespace Library.Api;


public sealed class Book
{
   public Guid Id { get; set; }
   public string Title { get; set; } = "";
   public string OriginalName { get; set; } = "";
   public string StoredName { get; set; } = "";
   public string Owner { get; set; } = "";
   public string Sha256 { get; set; } = "";
   public long Length { get; set; }
   public bool Published { get; set; }
}
public sealed class LibraryDb(DbContextOptions<LibraryDb> options) : DbContext(options)
{
   public DbSet<Book> Books => Set<Book>();
}
public sealed record UploadMetadata(string Title, string Filename, long Length);
/// <summary>Проверяет поля перед чтением и сохранением содержимого.</summary>
public sealed class MetadataValidator : AbstractValidator<UploadMetadata>
{
   public MetadataValidator()
   {
      RuleFor(x => x.Title).NotEmpty().MaximumLength(100)
```

```
            .Must(x => x is not null && !x.Any(char.IsControl));
        RuleFor(x => x.Filename).Must(FilePolicy.IsSafeName);
        RuleFor(x => x.Length).GreaterThan(0).LessThanOrEqualTo(FilePolicy.InputLimit);
    }
}
public sealed record StorageRoot(string Path);
/// <summary>Сохраняет только полностью проверенный набор в приватном карантине.</summary>
public sealed class UploadService(LibraryDb db, StorageRoot root, MetadataValidator validator)
{
    /// <summary>Выполняет валидацию, транзакцию метаданных и компенсацию файловой
записи.</summary>
    public async Task<Book[]> SaveAsync(string title, IFormFile file, string actor, CancellationToken ct)
    {
        var check = validator.Validate(new UploadMetadata(title, file.FileName, file.Length));
        if (!check.IsValid) throw new RejectedFile("Некорректные поля загрузки");
        using var input = file.OpenReadStream();
        var accepted = FilePolicy.Validate(file.FileName, FilePolicy.ReadLimited(input, FilePolicy.InputLimit));
        var rows = accepted.Select(x => new Book { Id = Guid.NewGuid(), Title = title,
            OriginalName = x.OriginalName, Owner = actor, StoredName = Guid.NewGuid().ToString("N") +
x.Extension,
            Length = x.Bytes.Length, Sha256 = Convert.ToHexString(SHA256.HashData(x.Bytes)), Published =
false }).ToArray();
        var written = new List<string>();
        await using var tx = await db.Database.BeginTransactionAsync(ct);
        try
        {
            for (int i = 0; i < rows.Length; i++)
            {
                string path = System.IO.Path.Combine(root.Path, "quarantine", rows[i].StoredName);
                await using var output = new FileStream(path, FileMode.CreateNew, FileAccess.Write);
                written.Add(path);
                await output.WriteAsync(accepted[i].Bytes, ct);
            }
            db.Books.AddRange(rows);
            await db.SaveChangesAsync(ct);
            await tx.CommitAsync(ct);
            return rows;
        }
        catch
        {
            await tx.RollbackAsync(CancellationToken.None);
            foreach (string path in written) File.Delete(path);
            throw;
        }
    }
}
```

src/Library.Api/BooksController.cs

```
using System.Security.Claims;
using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Mvc;
using Microsoft.EntityFrameworkCore;
namespace Library.Api;


/// <summary>Читает каталог с параметризованным LINQ и изоляцией карантина.</summary>
[ApiController]
[Route("api/books")]
[Authorize]
public sealed class BooksController(LibraryDb db, StorageRoot root) : ControllerBase
```

```
{
    /// <summary>Не раскрывает чужие неопубликованные документы.</summary>
    [HttpGet]
    public async Task<IActionResult> Get([FromQuery] string? query, CancellationToken ct)
    {
        string actor = User.FindFirstValue(ClaimTypes.NameIdentifier)!;
        bool librarian = User.IsInRole("Librarian");
        var books = db.Books.Where(x => x.Published || (librarian && x.Owner == actor));
        if (!string.IsNullOrWhiteSpace(query)) books = books.Where(x => x.Title.Contains(query));
        return Ok(await books.Select(x => new { x.Id, x.Title, x.OriginalName, x.Length,
            Status = x.Published ? "Published" : "Quarantined" }).ToArrayAsync(ct));
    }
    /// <summary>Выдаёт только опубликованный образец вне веб-каталога.</summary>
    [HttpGet("{id:guid}/file")]
    public async Task<IActionResult> Download(Guid id, CancellationToken ct)
    {
        var book = await db.Books.SingleOrDefaultAsync(x => x.Id == id && x.Published, ct);
        if (book is null) return NotFound();
        Response.Headers["X-Content-Type-Options"] = "nosniff";
        return PhysicalFile(Path.Combine(root.Path, "published", book.StoredName),
            "application/octet-stream", book.OriginalName);
    }
}
```